

Lezione 2: Esercitazione Pratica

Collezioni, Interfacce Funzionali e Generics in Java

Michele Gentile

7 Maggio 2026

0 Ripasso Teoria

A. Java Generics: Il "Contenitore Etichettato"

Concetto in breve

I Generics permettono di definire classi, interfacce e metodi che lavorano con tipi di dati specificati solo al momento dell'istanza.

Tip: Analogia del Trasloco

Immaginate dei traslochi. Se mettiamo tutto in scatole di cartone anonime, quando arriviamo nella casa nuova devi aprirle tutte per sapere cosa c'è dentro (questo è il "Casting" forzato). Se invece usi scatole con un'etichetta prestampata `<Libri>` o `<Piatti>`, sai già cosa puoi metterci dentro e cosa uscirà, evitando di rompere i piatti cercando di leggerli come libri.

Spiegazione Tecnica Dettagliata

Prima dei Generics (Java 5), le liste accettavano oggetti di tipo `Object`. Questo causava errori a runtime (`ClassCastException`). Con i Generics `<T>`, il compilatore controlla i tipi prima che il programma venga eseguito.

- **Type Parameter:** `<T>` (convenzione: T per Type, E per Element).
- **Wildcards:** `<?>` (qualsiasi tipo), `<? extends T>` (T o sottoclassi).

Listing 1: Esempio di Classe Generica

```
public class Scatola<T> {
    private T contenuto;

    public void inserisci(T oggetto) {
        this.contenuto = oggetto;
    }

    public T preleva() {
```

```

        return contenuto;
    }
}

// Utilizzo:
Scatola<String> scatolaLibri = new Scatola<>();
scatolaLibri.inserisci("Il Signore degli Anelli");
// scatolaLibri.inserisci(10); // ERRORE IN COMPILAZIONE!

```

⚠ Attenzione: Type Erasure

I Generics in Java esistono solo durante la **compilazione**. Una volta che il codice viene eseguito (runtime), i tipi generici vengono "cancellati" (*Type Erasure*) e sostituiti con `Object`.

È il motivo per cui, ad esempio, non si può scrivere `new T()` all'interno di una classe generica: poiché a runtime `T` scompare, Java non saprebbe quale costruttore chiamare.

B. java.util.function: Gli "Attrezzi Specializzati"

🔗 Concetto in breve

È un pacchetto che contiene interfacce con un solo metodo astratto (interfacce funzionali), pensate per essere usate con le espressioni Lambda.

💡 Tip: Analogia in Cucina

- **Predicate (Il Tester):** Il termometro. Gli dai qualcosa, ti dice "Vero" o "Falso" (es: è cotto?).
- **Function (Il Trasformatore):** Un frullatore. Gli dai la frutta, restituisce il succo.
- **Consumer (L'Utilizzatore):** Una pattumiera. Riceve e non restituisce nulla.
- **Supplier (Il Fornitore):** Una gallina. Non riceve input, ma ti dà un uovo.

Specifiche Tecniche

- **Predicate<T>:** Metodo `test(T t)`. Ritorna `boolean`.
- **Function<T, R>:** Metodo `apply(T t)`. Prende `T`, ritorna `R`.
- **Consumer<T>:** Metodo `accept(T t)`. Ritorna `void`.
- **Supplier<T>:** Metodo `get()`. Ritorna `T`.

Listing 2: Utilizzo Interfacce Funzionali

```

import java.util.function.*;

public class TestFunzioni {
    public static void main(String[] args) {

```

```

// La stringa e' piu' lunga di 10 caratteri?
Predicate<String> isLunga = s -> s.length() > 10;
// Restituisci la lunghezza della stringa
Function<String, Integer> contaCaratteri = s -> s.length();

System.out.println(isLunga.test("Informatica")); // true
System.out.println(contaCaratteri.apply("Java")); // 4
}
}

```

C. Strutture Dati: List vs Set

💡 Tip: Organizzazione della Memoria

ArrayList: È come una fila di sedie numerate in un cinema. Se sai il numero del posto (indice), arrivi subito. Ma se qualcuno vuole sedersi in mezzo, tutti quelli dopo devono spostarsi.

LinkedList: È come una caccia al tesoro. Ogni indizio ti dice dove si trova il prossimo. È facile aggiungere un indizio in mezzo, ma per trovare il decimo devi passare per tutti i precedenti.

HashSet: È come un sacchetto magico dove non possono esserci due oggetti identici. Non ti interessa l'ordine, ma trovi le cose all'istante.

TreeSet: È come una libreria perfettamente ordinata. Ogni volta che aggiungi un libro, lo metti subito nella posizione corretta per mantenere l'ordine.

Analisi delle Performance

- **ArrayList:** Accesso veloce ($O(1)$), inserimento lento in mezzo ($O(n)$).
- **LinkedList:** Accesso lento ($O(n)$), inserimento veloce in testa/coda ($O(1)$).
- **HashSet:** Ricerca e inserimento quasi istantanei ($O(1)$). No duplicati.
- **TreeSet:** Ordinamento automatico. Performance logaritmiche ($O(\log n)$).

Esercizi di verifica

🔗 Domande di Verifica

1. Se dichiaro `ArrayList<String>`, posso aggiungerci un `Integer`?
No, il compilatore lo impedisce.
2. Quale interfaccia funzionale useresti per stampare a video tutti gli elementi di una lista?
Consumer<T>, perché esegue un'azione (`System.out.println`) senza restituire nulla.
3. Devo inserire continuamente elementi in testa a una lista di un milione di record. Uso `ArrayList` o `LinkedList`?
LinkedList, perché aggiungere in testa è $O(1)$, mentre in `ArrayList` richiederebbe di spostare un milione di elementi ogni volta ($O(n)$).

1 Sistema di Gestione Dispositivi Smart

1. Si definisca una classe **Dispositivo** i cui oggetti sono costruiti con un **codice univoco** (String), un **nome** (es. "Lampada Soggiorno") e il **consumo energetico** (double).
2. Si crei una classe **SmartHome** che contiene un insieme di dispositivi utilizzando un **HashSet**.
3. Implementare i metodi **equals** e **hashCode** nella classe *Dispositivo* affinché due oggetti siano considerati duplicati se hanno lo stesso codice univoco.
4. Fornire una rappresentazione testuale chiara tramite il metodo **toString**.
5. Si implementi la sottoclasse **SmartHomeOrdinata** che utilizzi un **TreeSet** per mantenere i dispositivi ordinati in base al **consumo energetico** e, a parità di consumo, in base al **nome**.

🔗 Obiettivo: Gestione Set e Ordinamento

L'esercizio mira a consolidare la comprensione del contratto tra **equals/hashCode** per le collezioni basate su hash e l'interfaccia **Comparable** per le collezioni ordinate.

💡 Tip: Coerenza tra equals e compareTo

Per evitare comportamenti inattesi nel **TreeSet**, assicuratevi che **compareTo** restituisca 0 se e solo se **equals** restituisce **true** (ove possibile) o gestite esplicitamente i criteri di spareggio.

2 Pipeline di Elaborazione Stringhe

Creare una classe **TextProcessor** i cui oggetti gestiscono una sequenza di trasformazioni testuali.

1. La classe deve memorizzare una lista di funzioni (`java.util.function.Function<String, String>`).
2. Implementare un metodo **aggiungiFase**(`Function<String, String> f`) per inserire nuove trasformazioni nella pipeline.
3. Implementare il metodo **elabora**(`String input`) che applica in sequenza tutte le funzioni all'input e restituisce il risultato finale.
4. Nel **main**, istanziare la classe e aggiungere le seguenti fasi usando **lambda** o **method references**:
 - Rimozione degli spazi bianchi iniziali e finali (**trim**).
 - Conversione di tutto il testo in maiuscolo.
 - Sostituzione di tutti gli spazi interni con il carattere underscore (**_**).
 - Concatenazione del prefisso "[PROCESSED] " all'inizio della stringa.

🔗 Obiettivo: Interfacce Funzionali e Pipeline

Sperimentare la composizione di comportamenti tramite interfacce funzionali standard del package `java.util.function`.

3 Utility per Liste Generiche

Implementare una classe **ListUtility** contenente i seguenti metodi statici generici:

1. **filtra**: riceve in input una lista di tipo `T` e un `Predicate<T>`. Restituisce una nuova lista contenente solo gli elementi che soddisfano il predicato.
2. **trovaMinimo**: riceve in input una lista di elementi di tipo generico `T` e ne restituisce il valore minimo.

⚠️ Attenzione: Vincoli sui Generics

Per il metodo `trovaMinimo`, è obbligatorio utilizzare il costrutto `<T extends Comparable<T>` per garantire che gli elementi siano confrontabili tra loro.

4 Gestione Log Eventi

Si vuole gestire un sistema di log cronologico.

1. Si definisca la classe **LogEntry** con un **timestamp** (`LocalDateTime`) e un **messaggio** (`String`).
2. Implementare l'interfaccia `Comparable` per ordinare i log in base al tempo (dal più recente al meno recente).
3. Si crei una classe **RegistroEventi** che utilizza un `TreeSet` per memorizzare i log.
4. Implementare un metodo `eseguiAzioneSuLog(Consumer<LogEntry> action)` che applica l'azione fornita a ogni log nel registro.

💡 Tip: Java Time API

Per gestire il tempo in Java moderno, utilizzate `java.time.LocalDateTime`. Per l'ordinamento decrescente, prestate attenzione all'ordine degli operandi nel metodo `compareTo`.